Praktikum 1 - Matakuliah Pilihan Web (API Development)
Program Studi: Teknik Informatika

Lakukan praktikum dibawah ini, dan buat screenshot untuk pembuktian mengerjakan setiap poin dengan mengisi tabel dibawah, kemudian tunjukan hasil akhir dari men-share repository github yang telah dibuat.

A. Setup Laravel dan Mysql

1. Buat project laravel baru dengan composer dengan nama laravel_api (Gunakan versi 12.x atau yang lebih baru)
2. Mempersiapkan database dan buat database baru bernama dblaravelapi
3. Update .env dan hubungkan dengan nama database dblaravelapi
4. Lakukan migrasi dengan menggunakan php artisan migrate
5. Pastikan laravel sudah terhubung dengan tabel dengan mengecek tabel yang dibuat laravel dalam mysql

B. Membuat Controller

1. Buatlah sebuah controller dengan nama Api/ProductController
   php artisan make:controller Api/ProductController
   (Pastikan ada di dalam folder Api)
   Pada bootstrap/app.php tambahkan setelah code web: __DIR
   api: __DIR__.'/../routes/api.php',

```
return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
```

2. Tambahkan Routes baru di router/api.php
   Route::get('/products', [ProductController::class, 'index']);
   (Apabila file api.php belum ada, clone web.php dan hapus routes "/")
3. Test API dengan mengunjungi localhost:8000/api/products

C. Gunakan database dan Model

1. Pastikan database sudah terkoneksi dengan laravel.
2. Buatkan sebuah tabel dengan nama tblproducts dengan field (id, nama, deskripsi, foto, dan harga) untuk tipe data menyesuaikan.

3. Tambahkan minimal 10 data untuk produk dengan nama2 product elektronik seperti tv, kulkas, ac, dan lain sebagainya.
4. Buat model bernama Product
   php artisan make:model Product
5. Hubungkan model Product dengan tblproduct dengan mengisi pada class Product

```
class Product extends Model
{
    protected $table = 'tblproducts';
    protected $primaryKey = 'id';
    public $timestamps = false;

    // Field yang boleh diisi
    protected $fillable = [
        'nama',
        'deksripsi',
        'foto',
        'harga',
    ];

}
```

6. Pada Product controller ubah dalam fungsi index, dapatkan data dari model Product
   Dengan menambahkan $products = Product:all();
   Dan lakukan return data => $products

D. Membuat CRUD standar API Resource

1. Ubah api.php dengan api Resource yang sesuai standar laravel

```
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\ProductController;

// Route::get('/products', [ProductController::class, 'index']);

Route::apiResource('products', ProductController::class);
```

Dengan apiResource kamu tidak perlu menambahkan routes satu persatu untuk ProductController seperti create, update, delete, search dan view. Namun ada syarat penamaan fungsi agar routes ini bekerja yaitu **index, show, store, update, dan destroy.**

2. Ubah ProductController untuk fungsi-fungsi berikut **index, show, store, update, dan destroy.**

```php
// 🔹 GET /products
public function index()
{
    $products = Product::all();

    return response()->json([
        'status' => true,
        'data' => $products
    ]);
}
```

Gambar 04: Fungsi Index pada ProductsController

```php
// 🔹 GET /products/{id}
public function show($id)
{
    $product = Product::find($id);

    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }

    return response()->json([
        'status' => true,
        'data' => $product
    ]);
}
```

Gambar 05: Fungsi Show pada ProductsController

```php
// 🔹 POST /products
public function store(Request $request)
{
    $request->validate([
        'nama' => 'required',
        'deksripsi' => 'required',
        'harga' => 'required|numeric'
    ]);

    $product = Product::create($request->all());

    return response()->json([
        'status' => true,
        'message' => 'Product berhasil ditambahkan',
        'data' => $product
    ]);
}
```

Gambar 06: Fungsi Store pada ProductsController

```php
// 🔹 PUT /products/{id}
public function update(Request $request, $id)
{
    $product = Product::find($id);
    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }
    $request->validate([
        'nama' => 'required',
        'deksripsi' => 'required',
        'harga' => 'required|numeric'
    ]);
    $product->update($request->all());
    return response()->json([
        'status' => true,
        'message' => 'Product berhasil diupdate',
        'data' => $product
    ]);
}
```

Gambar 07: Fungsi Update pada ProductsController

```php
// 🔹 DELETE /products/{id}
public function destroy($id)
{
    $product = Product::find($id);

    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }

    $product->delete();

    return response()->json([
        'status' => true,
        'message' => 'Product berhasil dihapus'
    ]);
}
```

Gambar 08: Fungsi Destroy pada ProductsController

E. API Documentation

1. Install Scramble dengan menggunakan composer
   ```
   composer require dedoc/scramble
   ```

2. Masukan ke vendor laravel project anda
   ```
   php artisan vendor:publish
   --provider="Dedoc\Scramble\ScrambleServiceProvider"
   ```

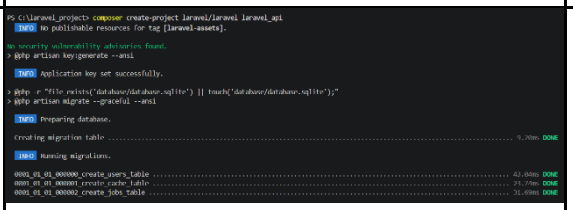
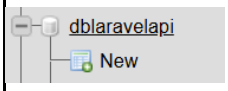
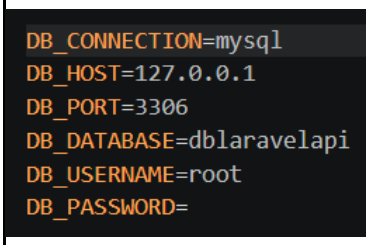
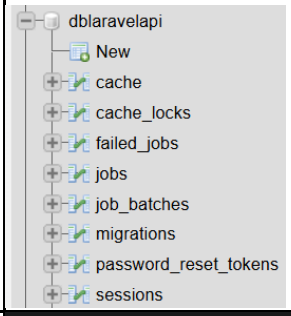
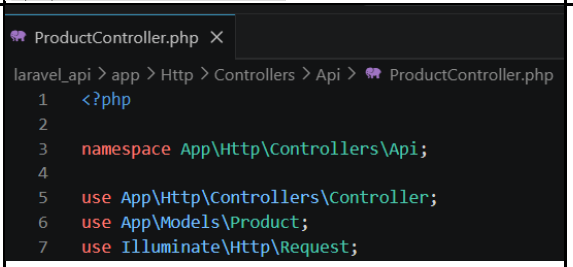
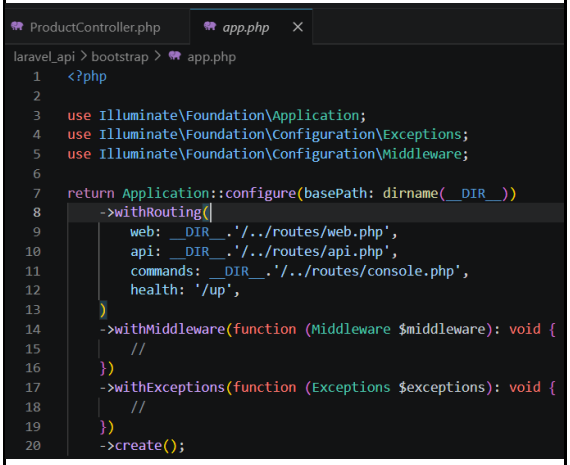
3. Jalankan server (artisan serve)
4. Akses API documentation pada browser ke http://localhost:8000/docs/api
5. Untuk menambahkan deskripsi setiap API Endpoints bisa menggunakan DocBlock diatas fungsi dalam controller
6. Menambahkan DocBlock (Title dan Description)

```php
class ProductController extends Controlle
{

/**
 * Ambil semua produk.
 *
 * Endpoint ini mengembalikan seluruh data produk yang tersedia.
 */

    // 🔹 GET /products
    public function index()
    {
        $products = Product::all();

        return response()->json([
            'status' => true,
            'data' => $products
        ]);
    }
```

Gambar 08: DocBlock untuk title node index dan deksripsi

7. Refresh browser ke http://localhost:8000/docs/api
8. Tambahkan Title dan Deskripsi pada semua Endpoints

Hasil Pengerjaan

No.	Instruksi	Screenshot	Kendala/Saran
A.	Installasi dan Konfigurasi		
1.	1. Buat project Laravel baru menggunakan Composer dengan nama *laravel_api* (gunakan versi 12.x atau yang lebih baru). 2. Mempersiapkan database dan membuat database baru bernama *dblaravelapi*. 3. Memperbarui file *.env* dan menghubungkannya dengan database bernama *dblaravelapi*. 4. Melakukan migrasi database menggunakan perintah php artisan migrate serta memastikan Laravel telah terhubung dengan database dengan cara mengecek tabel yang dibuat pada MySQL.	   	Tidak ada
2.	1. Membuat controller dengan nama *Api/ProductController* menggunakan perintah php artisan make:controller Api/ProductController, serta memastikan controller tersebut berada di dalam folder *Api*. Selanjutnya, melakukan konfigurasi pada file bootstrap/app.php dengan menambahkan kode routing API setelah konfigurasi web, yaitu api: __DIR__.'/../routes/api.php', 2. Menambahkan route baru pada file routes/api.php	 	Tidak ada

		menggunakan kode Route::get('/products', [ProductController::class, 'index']);. Jika file api.php belum tersedia, maka dapat dibuat dengan menyalin file web.php dan menghapus route default "/". Setelah itu, dilakukan pengujian API dengan mengakses endpoint http://localhost:8000/api/products menggunakan browser atau aplikasi seperti Postman.	*api.php*
			laravel_api > routes > api.php
			`<?php`
			
			`use Illuminate\Support\Facades\Route;`
			`use App\Http\Controllers\Api\ProductController;`
			
			`Route::apiResource('products', ProductController::class);`
			
			127.0.0.1:8000/api/products
			Pretty-print
			{
			"status": true,
			"data": [
			{
			"id": 1,
			"nama": "TV LED Samsung 43 Inch",
			"deskripsi": "Televisi LED ukuran 43 inch dengan resolusi Full HD",
			"foto": "tv-samsung.jpg",
			"harga": "4200000.00",
			"created_at": "2026-04-06T14:24:08.000000Z",
			"updated_at": "2026-04-06T14:24:08.000000Z"
			},
			{
			"id": 2,
			"nama": "Kulkas LG 2 Pintu",
			"deskripsi": "Kulkas dua pintu hemat listrik dan kapasitas besar",
			"foto": "kulkas-lg.jpg",
			"harga": "3500000.00",
			"created_at": "2026-04-06T14:24:08.000000Z",
			"updated_at": "2026-04-06T14:24:08.000000Z"
			},
			{
			"id": 3,
			"nama": "AC Panasonic 1 PK",
			"deskripsi": "AC hemat energi dengan pendinginan cepat",
			"foto": "ac-panasonic.jpg",
			"harga": "3200000.00",
			"created_at": "2026-04-06T14:24:08.000000Z",
			"updated_at": "2026-04-06T14:24:08.000000Z"
			},
			{
			"id": 4,
			"nama": "Mesin Cuci Sharp",
			"deskripsi": "Mesin cuci otomatis kapasitas 7 kg",
			"foto": "mesin-cuci-sharp.jpg",
			"harga": "2800000.00",
			"created_at": "2026-04-06T14:24:08.000000Z",
			"updated_at": "2026-04-06T14:24:08.000000Z"
			},
			{
			"id": 5,
			"nama": "Laptop ASUS VivoBook",
			"deskripsi": "Laptop untuk kerja dan kuliah dengan performa stabil",
			"foto": "laptop-asus.jpg",
			"harga": "7800000.00",
			"created_at": "2026-04-06T14:24:08.000000Z",
			"updated_at": "2026-04-06T14:24:08.000000Z"
			},
			{
			"id": 6,
			"nama": "Smartphone Xiaomi Redmi Note",
			"deskripsi": "HP dengan kamera jernih dan baterai tahan lama",
			"foto": "xiaomi-redmi.jpg",
			"harga": "2900000.00",
			"created_at": "2026-04-06T14:24:08.000000Z",
			"updated_at": "2026-04-06T14:24:08.000000Z"
			},

3.	1. Pastikan database sudah terkoneksi dengan Laravel.
	2. Buatlah sebuah tabel dengan nama *tblproducts* dengan field *(id, nama, deskripsi, foto, dan harga)*, serta sesuaikan tipe data masing-masing field.
	3. Tambahkan minimal 10 data produk, misalnya produk elektronik seperti TV, kulkas, AC, dan lain sebagainya.
	4. Buat model dengan nama *Product* menggunakan perintah php artisan make:model Product.
	5. Hubungkan model *Product* dengan tabel *tblproducts* dengan melakukan konfigurasi pada class *Product*.
	6. Pada *ProductController*, ubah fungsi index untuk mengambil data dari model *Product* dengan menambahkan $products = Product::all();, kemudian kembalikan data tersebut dengan return $products;.
		Browse Structure SQL Search Insert Export Import Privileges Operations Triggers
		Showing rows 0 - 9 (10 total, Query took 0.0015 seconds.)
		SELECT * FROM `tblproducts`
		Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
		Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None
		Extra options
		+ Options
		id nama deskripsi foto harga created_at updated_at
		1 TV LED Samsung 43 Inch Televisi LED ukuran 43 inch dengan resolusi Full H tv-samsung.jpg 4200000.00 2026-04-06 14:24:08 2026-04-06 14:24:08
		2 Kulkas LG 2 Pintu Kulkas dua pintu hemat listrik dan kapasitas besar kulkas-lg.jpg 3500000.00 2026-04-06 14:24:08 2026-04-06 14:24:08
		3 AC Panasonic 1 PK AC hemat energi dengan pendinginan cepat ac-panasonic.jpg 3200000.00 2026-04-06 14:24:08 2026-04-06 14:24:08
		4 Mesin Cuci Sharp Mesin cuci otomatis kapasitas 7 kg mesin-cuci-sharp.jpg 2800000.00 2026-04-06 14:24:08 2026-04-06 14:24:08
		5 Laptop ASUS VivoBook Laptop untuk kerja dan kuliah dengan performa stab laptop-asus.jpg 7800000.00 2026-04-06 14:24:08 2026-04-06 14:24:08
		6 Smartphone Xiaomi Redmi Note HP dengan kamera jernih dan baterai tahan lama xiaomi-redmi.jpg 2900000.00 2026-04-06 14:24:08 2026-04-06 14:24:08
		7 Blender Philips Blender multifungsi untuk kebutuhan dapur blender-philips.jpg 850000.00 2026-04-06 14:24:08 2026-04-06 14:24:08
		8 Rice Cooker Miyako Pemanak nasi otomatis dengan timer rice-cooker-miyako.jpg 600000.00 2026-04-06 14:24:08 2026-04-06 14:24:08
		9 Kipas Angin Cosmos Kipas angin berdiri dengan 3 level kecepatan kipas-cosmos.jpg 350000.00 2026-04-06 14:24:08 2026-04-06 14:24:08
		10 Microwave Electrolux Microwave praktis untuk memanaskan makanan microwave-electrolux.jpg 1500000.00 2026-04-06 14:24:08 2026-04-06 14:24:08
		Check all With selected: Edit Copy Delete Export
		Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None
		Query results operations
		Console
		
		Product.php
		laravel_api > app > Models > Product.php
		`<?php`
		
		`namespace App\Models;`
		
		`use Illuminate\Database\Eloquent\Model;`
		
		`class Product extends Model`
		`{`
		`protected $table = 'tblproducts';`
		`protected $primaryKey = 'id';`
		
		`protected $fillable = [`
		`'nama',`
		`'deskripsi',`
		`'foto',`
		`'harga',`
		`];`
		`}`
		
		ProductController.php
		laravel_api > app > Http > Controllers > Api > ProductController.php
		`<?php`
		
		`namespace App\Http\Controllers\Api;`
		
		`use App\Http\Controllers\Controller;`
		`use App\Models\Product;`
		`use Illuminate\Http\Request;`
		
		`class ProductController extends Controller`
		`{`
		`/**`
		`* Ambil semua produk.`
		`*`
		`* Endpoint ini mengembalikan seluruh data produk yang tersedia.`
		`*/`
		`// GET /products`
		`public function index()`
		`{`
		`$products = Product::all();`
		
		`return response()->json([`
		`'status' => true,`
		`'data' => $products`
		`]);`
		`}`
			Tidak ada

4.	1. Ubah file api.php dengan menggunakan API Resource sesuai standar Laravel. 2. Ubah *ProductController* dengan menambahkan fungsi-fungsi berikut: index, show, store, update, dan destroy.	

api.php

laravel_api > routes > api.php

```php
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\ProductController;

Route::apiResource('products', ProductController::class);
```

```php
/**
 * Ambil semua produk.
 *
 * Endpoint ini mengembalikan seluruh data produk yang tersedia.
 */
// GET /products
public function index()
{
    $products = Product::all();

    return response()->json([
        'status' => true,
        'data' => $products
    ]);
}
```

```php
/**
 * Ambil detail produk.
 *
 * Endpoint ini menampilkan data satu produk berdasarkan id.
 */
// GET /products/{id}
public function show($id)
{
    $product = Product::find($id);

    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }

    return response()->json([
        'status' => true,
        'data' => $product
    ]);
}
```

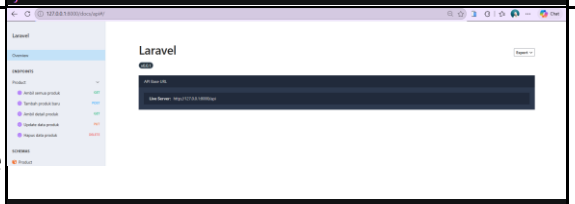
```php
/**
 * Tambah produk baru.
 *
 * Endpoint ini digunakan untuk menyimpan data produk baru ke database.
 */
// POST /products
public function store(Request $request)
{
    $request->validate([
        'nama' => 'required',
        'deskripsi' => 'required',
        'harga' => 'required|numeric',
        'foto' => 'nullable'
    ]);

    $product = Product::create($request->all());

    return response()->json([
        'status' => true,
        'message' => 'Product berhasil ditambahkan',
        'data' => $product
    ]);
}
```

```php
/**
 * Update data produk.
 *
 * Endpoint ini digunakan untuk mengubah data produk berdasarkan id.
 */
// PUT /products/{id}
public function update(Request $request, $id)
{
    $product = Product::find($id);

    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }

    $request->validate([
        'nama' => 'required',
        'deskripsi' => 'required',
        'harga' => 'required|numeric',
        'foto' => 'nullable'
    ]);

    $product->update($request->all());

    return response()->json([
        'status' => true,
        'message' => 'Product berhasil diupdate',
        'data' => $product
    ]);
}
```

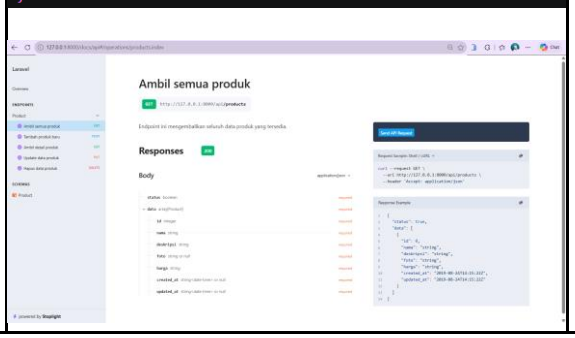
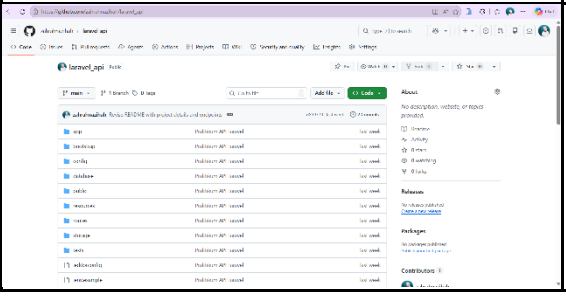
```php
/**
 * Hapus data produk.
 *
 * Endpoint ini digunakan untuk menghapus data produk berdasarkan id.
 */
// DELETE /products/{id}
public function destroy($id)
{
    $product = Product::find($id);

    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }

    $product->delete();

    return response()->json([
        'status' => true,
        'message' => 'Product berhasil dihapus'
    ]);
}
```

5.

1. Install Scramble menggunakan Composer dengan perintah: composer require dedoc/scramble
2. Publikasikan vendor pada project Laravel dengan perintah: php artisan vendor:publish --provider="Dedoc\Scramble\ScrambleServiceProvider"
3. Jalankan server Laravel dengan perintah: php artisan serve
4. Akses dokumentasi API melalui browser pada alamat: http://localhost:8000/docs/api
5. Untuk menambahkan deskripsi pada setiap endpoint API, gunakan




```php
/**
 * Ambil semua produk.
 *
 * Endpoint ini mengembalikan seluruh data produk yang tersedia.
 */
// GET /products
public function index()
{
    $products = Product::all();

    return response()->json([
        'status' => true,
        'data' => $products
    ]);
}
```



	DocBlock di atas fungsi pada controller. 6. Tambahkan DocBlock berupa Title dan Description pada setiap endpoint. 7. Refresh browser untuk melihat perubahan dokumentasi API. 8. Pastikan setiap endpoint memiliki Title dan Description yang sesuai		
B.	Github dan Viscode		
1.	https://github.com/zahrahnazihah/laravel_api.git